HO4. Solve Problem 7.3 Predicting Housing Median Prices in your textbook

In [7]:
```
import matplotlib.pyplot as plt
import mlba
import pandas as pd
import numpy as np
from sklearn import preprocessing
from sklearn.compose import ColumnTransformer # apply different preprocessir
from sklearn.model_selection import train_test_split, cross_val_score, Leave
from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor
from sklearn.pipeline import Pipeline
from sklearn.model_selection import KFold
%matplotlib inline
```

In [2]:
```
#VB67
# Loading the dataset
boston_df = mlba.load_data('BostonHousing.csv')

# Define predictors (first 12 columns) and outcome (MEDV)
predictors = boston_df.columns[:12]
outcome = 'MEDV'

# Splitting into training (60%) and holdout (40%)
train_df, holdout_df = train_test_split(boston_df, test_size=0.4, random_sta

# Separate X and y
train_X = train_df[predictors]
train_y = train_df[outcome]
holdout_X = holdout_df[predictors]
holdout_y = holdout_df[outcome]

# Standardize predictors
scaler = preprocessing.StandardScaler()
train_X_scaled = scaler.fit_transform(train_X)
holdout_X_scaled = scaler.transform(holdout_X)
```

A. Perform a Math equation k-NN prediction with all 12 predictors (ignore the CAT. MEDV column), trying values of Math equation from 1 to 5. Use 5-fold cross-validation to find the best Math equation. Make sure to normalize the data. What is the best Math equation? What does it mean?

In [10]:
```
#VB67
best_k = None
best_score = -np.inf
results = []

# Try k values from 1 to 5
for k in range(1, 6):
    knn = KNeighborsRegressor(n_neighbors=k)
```

```
    scores = cross_val_score(knn, train_X_scaled, train_y, cv=5, scoring='ne
    mean_score = scores.mean()
    results.append({'k': k, 'mean_R2': mean_score})

    if mean_score > best_score:
        best_score = mean_score
        best_k = k

# Converting results to DataFrame
pd.DataFrame(results)
```

Out[10]:

	k	mean_R2
0	1	-33.455236
1	2	-25.972809
2	3	-25.947407
3	4	-25.027534
4	5	-25.970976

The best value of k=4

B. Predict the MEDV for a tract with the following information, using the best Math equation

In [11]:

```
#VB67
# New tract data
new_tract = pd.DataFrame([{
    'CRIM': 0.2, 'ZN': 0, 'INDUS': 7, 'CHAS': 0, 'NOX': 0.538,
    'RM': 6, 'AGE': 62, 'DIS': 4.7, 'RAD': 4, 'TAX': 307,
    'PTRATIO': 21, 'LSTAT': 10
}])

# Scale new tract
new_scaled = scaler.transform(new_tract)

# Fitting k-NN with best k
knn_best = KNeighborsRegressor(n_neighbors=best_k)
knn_best.fit(train_X_scaled, train_y)

# Predicting MEDV
predicted_MEDV = knn_best.predict(new_scaled)
predicted_MEDV[0]
```

Out[11]: `np.float64(18.7)`

Predicted MEDV for new tract is $18.70k

C. If we used the above Math equation-NN algorithm to score the training data, what would be the error of the training set?

In [12]:
```
#VB67
# Predicting on training data
train_pred = knn_best.predict(train_X_scaled)

# Computing training Mean Squared Error (MSE)
from sklearn.metrics import mean_squared_error
train_error = mean_squared_error(train_y, train_pred)
train_error
```

Out[12]:
```
14.431256188118809
```

The error of training set is 14.431

D. If the purpose is to predict MEDV for several thousands of new tracts, what would be the disadvantage of using Math equation-NN prediction? List the operations that the algorithm goes through to produce each prediction.

VB67 - Disadvantage: As k-NN is a lazy learner and has very high computational expenses specially for large-scale predictions, It handles critical computations during the prediction phase and store all the data used for training. It has to normalize the data and estimate the distances to each training data, determine the k nearest neighbors,and aggregate their MEDV values for every new tract. Therefore, while preparing thousands of more cases, the process becomes slower and inefficient.

In [ ]: